

# Bài 6: Preliminary Design Review (PDR) — Kiểm duyệt thiết kế sơ bộ

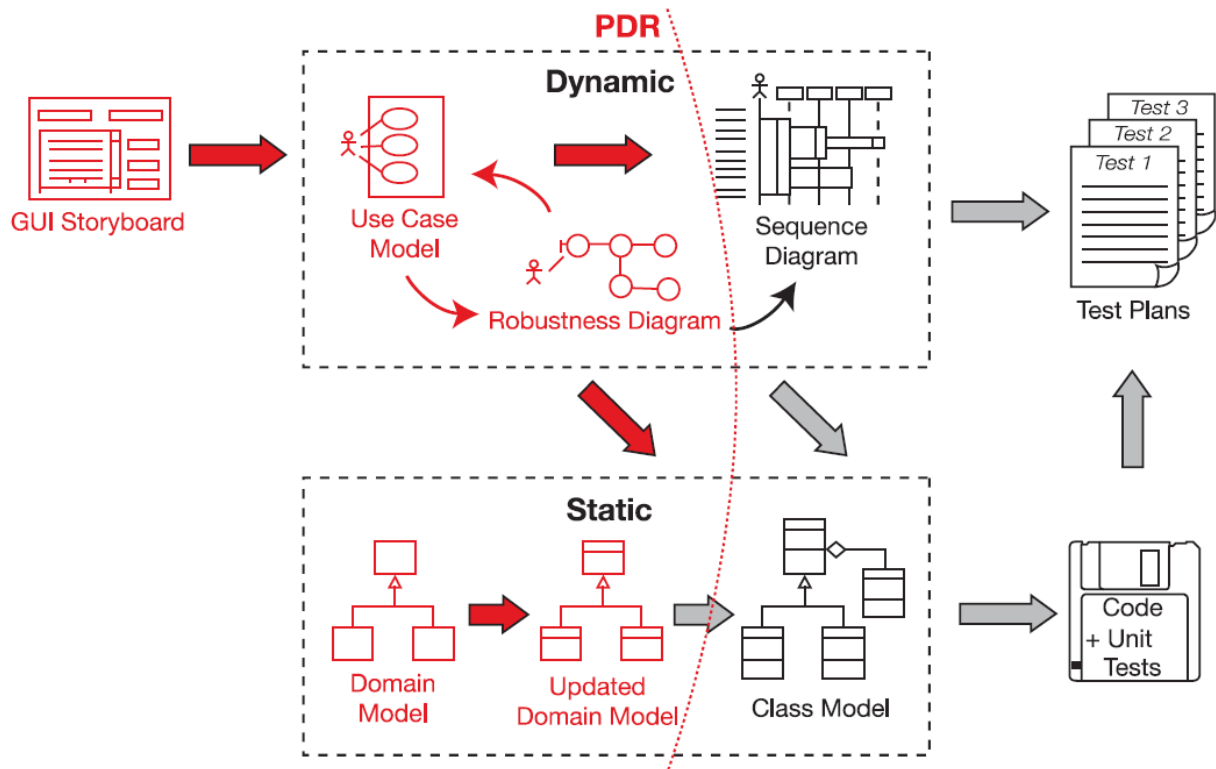
## Nội dung chính

Bài này trình bày giai đoạn Preliminary Design Review (PDR) trong quy trình ICONIX — cột mốc then chốt nhằm đảm bảo tính nhất quán và đầy đủ giữa Use Case, Domain Model và Robustness Diagram, trước khi bước sang thiết kế chi tiết (sequence diagram, class diagram). Đây là bước kiểm duyệt tập thể (review session) có sự tham gia của cả nhóm kỹ thuật lẫn khách hàng, giúp phát hiện những lỗi hổng logic, thiếu sót trong xử lý alternate flow, các vi phạm quy tắc robustness, và đảm bảo tất cả thực thể miền đã được phản ánh chính xác vào mô hình. Nếu Robustness Analysis là cầu nối giữa hành vi và thiết kế, thì PDR chính là trạm kiểm soát bắt buộc trên cây cầu đó.

## Mục tiêu cần đạt

Sau khi học xong bài này, các bạn sẽ:

- Hiểu rõ vai trò và ý nghĩa của PDR trong quy trình ICONIX, và lý do vì sao đây là một bước không thể bỏ qua.
- Nắm vững Top 10 PDR Guidelines — bộ nguyên tắc thực hành chuẩn giúp kiểm duyệt có hệ thống.
- Áp dụng được “Highlighter Test” để kiểm tra tính khớp giữa use case và sơ đồ robustness.
- Thực hiện được PDR 6 bước kiểm duyệt sơ đồ và văn bản, đảm bảo logic hệ thống sẵn sàng bước vào giai đoạn thiết kế chi tiết.
- Nhận diện được các lỗi phổ biến trong use case và robustness diagram thông qua ví dụ thực tế Internet Bookstore.
- Hiểu mối quan hệ giữa PDR, Technical Architecture và Detailed Design.



## 6.1 Bối cảnh và vấn đề thực tiễn

Sau khi nhóm phát triển đã hoàn thành các bước quan trọng:

- Xây dựng Domain Model sơ khởi
- Viết Use Case với các luồng chính và phụ
- Vẽ Robustness Diagram để ràng buộc hành vi – đối tượng

... thì một câu hỏi lớn xuất hiện:

“Liệu mô hình của chúng ta có thực sự sẵn sàng để bước sang giai đoạn thiết kế chi tiết chưa?”

Trong thực tế, rất nhiều dự án mắc phải 3 nhóm lỗi phổ biến ở giai đoạn này:

1. Mô hình không khớp nhau: Use case nói một đằng, Robustness diagram vẽ một nẻo, Domain model thì thiếu thực thể hoặc tên khác nhau.
2. Thiếu alternate flow hoặc xử lý đặc biệt: Các kịch bản lỗi và ngoại lệ bị bỏ sót, dẫn đến “bug logic” khi thiết kế chi tiết hoặc lập trình.
3. Thiếu sự tham gia của stakeholders: Việc review mô hình chỉ do nhóm kỹ thuật thực hiện, dẫn đến hiểu sai yêu cầu thật từ khách hàng.

Nếu không có một bước kiểm duyệt có hệ thống, nhóm dễ rơi vào tình trạng: “tưởng rằng mô hình đã xong”, nhưng khi vào thiết kế chi tiết thì phát hiện mâu thuẫn, phải sửa tới sửa lui, mất thời gian và tốn công sức gấp nhiều lần.

Chính vì vậy, ICONIX giới thiệu Preliminary Design Review (PDR) — một buổi kiểm duyệt tập trung, có quy trình, có nguyên tắc, có công cụ cụ thể để kiểm tra sự nhất quán, đầy đủ và rõ ràng của mô hình

## 6.2 Vị trí của PDR trong ICONIX Process

PDR diễn ra ngay sau Robustness Analysis, trước khi nhóm bắt đầu Detailed Design (sequence diagrams, class diagrams). Đây là “gateway” – cổng kiểm soát bắt buộc: chỉ khi mô hình qua được bước này thì nhóm mới được phép bước sang thiết kế chi tiết.

Hình 6-8 trong sách minh họa rõ vị trí của PDR trong dòng thời gian ICONIX:

- Requirements Review → Domain Model + Use Case + Storyboard
- Robustness Analysis (liên kết hành vi–đối tượng)
- PDR (kiểm duyệt mô hình)
- Sequence Diagram & Class Design (detailed design)

Tại sao bước này quan trọng? Bởi vì đây là lần cuối cùng khách hàng có tiếng nói trực tiếp vào mô hình. Sau PDR, toàn bộ phần còn lại (thiết kế chi tiết, lập trình, kiểm thử) thuộc về nhóm kỹ thuật. Nếu mô hình có vấn đề mà không được phát hiện ở đây, hậu quả sẽ kéo dài đến cuối dự án.

## 6.3 Mục đích và thành phần tham gia PDR

PDR có 3 mục tiêu chính

1. Đảm bảo sự nhất quán giữa:
  - Use case text
  - Robustness diagram
  - Domain model
2. Kiểm tra độ đầy đủ:
  - Tất cả alternate flow đã được mô tả và có hành vi xử lý rõ ràng
  - Các entity đã có đủ thuộc tính
  - Dòng dữ liệu giữa boundary ↔ entity có thể truy vết được
3. Xem xét khả năng tương thích với Technical Architecture (TA):
  - Mô hình có phù hợp với kiến trúc hệ thống đã chọn?
  - Có điểm nghẽn kỹ thuật tiềm ẩn không?

### Thành phần tham gia

Tương tự Requirements Review, PDR cần cả hai nhóm:

- Non-technical: đại diện khách hàng, marketing, người dùng chính.
- Technical: developer, kiến trúc sư, BA, tester.

PDR cũng là lần cuối khách hàng có thể góp ý ở cấp mô hình, trước khi dự án chuyển hoàn toàn sang tay nhóm kỹ thuật

## 6.4 Top 10 PDR Guidelines

Tác giả ICONIX đã tổng hợp **10 nguyên tắc vàng** giúp nhóm kiểm duyệt PDR một cách có hệ thống, không bỏ sót.

| #  | Guideline                                                            | Ý nghĩa thực tiễn                                |
|----|----------------------------------------------------------------------|--------------------------------------------------|
| 10 | Use Highlighter Test để đảm bảo use case khớp với robustness diagram | Kiểm tra từng câu → từng phần sơ đồ              |
| 9  | Đảm bảo mọi entity trên sơ đồ có trong domain model                  | Không bỏ sót thực thể phát hiện trong robustness |
| 8  | Truy vết được luồng dữ liệu giữa entity ↔ screen                     | Kiểm tra input/output có đủ thuộc tính           |
| 7  | Không bỏ alternate course và phải mô tả hành vi đầy đủ               | Alternate chiếm nửa độ phức tạp hệ thống         |
| 6  | Mỗi use case phải bao phủ cả user + system dialogue                  | Không chỉ mô tả hành động người dùng             |
| 5  | Không vi phạm quy tắc cú pháp robustness                             | Actor–boundary–controller–entity rõ ràng         |
| 4  | Bao gồm cả technical và non-technical trong buổi review              | Đảm bảo mọi bên hiểu chung một mô hình           |
| 3  | Đặt use case trong ngữ cảnh object model & GUI                       | Giải quyết mơ hồ về tên màn hình & class         |
| 2  | Không trôi sang detailed design                                      | Giữ đúng mức conceptual                          |
| 1  | Thực hiện 6 bước kiểm duyệt sơ đồ                                    | Kiểm tra toàn diện để đảm bảo sẵn sàng thiết kế  |

## 6.5 Phân tích chi tiết Top 10 Guidelines với ví dụ Internet Bookstore

Để hiểu rõ ý nghĩa thực tế của 10 nguyên tắc PDR, chúng ta sẽ lần lượt phân tích từ guideline #10 đến #1.

## #10. Highlighter Test — công cụ kiểm tra đơn giản nhưng hiệu quả

Một trong những mẹo thực hành thú vị nhất của ICONIX là “59-cent Highlighter Test”. Bạn in hoặc chiếu use case text và robustness diagram cạnh nhau. Dùng bút dạ để:

- Tô sáng từng câu trong use case
- Tô sáng phần tương ứng trên sơ đồ robustness. Khi toàn bộ use case được highlight, sơ đồ cũng phải được highlight hoàn toàn — và ngược lại.

Nếu phát hiện câu nào trong use case không có phần tương ứng trên sơ đồ, hoặc phần nào trên sơ đồ không khớp câu nào trong use case → đó chính là điểm lệch cần chỉnh sửa. Highlighter Test cực kỳ hiệu quả trong các buổi review tập thể, giúp khách hàng và lập trình viên có chung ngôn ngữ kiểm tra mà không cần đi vào chi tiết kỹ thuật.

## #9. Đảm bảo mọi entity trên sơ đồ có trong Domain Model

Một trong những giá trị lớn nhất của PDR là nó giúp kiểm tra sự ăn khớp giữa Robustness Diagram và Domain Model. Trong quá trình phân tích robustness, nhóm có thể phát hiện các entity mới (object discovery). Tuy nhiên, nhiều nhóm lại quên cập nhật domain model tương ứng. Điều này dẫn đến sự lệch pha giữa các mô hình, gây lỗi ở giai đoạn thiết kế chi tiết.

Ví dụ: Trong use case “Write Customer Review”, khi vẽ sơ đồ robustness, nhóm phát hiện entity “ReviewModerationQueue” (hàng đợi kiểm duyệt review). Thực thể này ban đầu không có trong domain model. → Trong PDR, guideline #9 buộc nhóm phải đối chiếu toàn bộ danh sách entity trong sơ đồ với domain model, bổ sung những entity còn thiếu để đảm bảo sự thống nhất

## #8. Truy vết được luồng dữ liệu giữa entity ↔ screen

Một sơ đồ robustness không chỉ đơn giản là vẽ đối tượng và mũi tên, mà còn phải ngầm chứa luồng dữ liệu. Guideline #8 yêu cầu nhóm kiểm duyệt đảm bảo rằng:

- Mỗi boundary (screen/page) có thể hiển thị đủ dữ liệu từ entity tương ứng.
- Mỗi entity có thể cung cấp thông tin cần thiết cho boundary.
- Không có dữ liệu “trôi nổi” — thuộc tính nào xuất hiện trên màn hình đều phải đến từ một entity cụ thể

Ví dụ: Trang “BookDetailsPage” cần hiển thị ISBN, Title, Price, AverageRating... → Tất cả các thuộc tính này phải tồn tại trong entity “Book” hoặc các entity liên quan (như “Review”).  
Nếu trang hiển thị “Number of Reviews” mà domain model không có thuộc tính nào lưu thông tin này → PDR sẽ phát hiện ra và nhóm sẽ phải bổ sung vào entity tương ứng.

## #7. Không bỏ alternate course và phải mô tả hành vi đầy đủ

Một thống kê thú vị từ các dự án ICONIX: khoảng 50% logic hệ thống nằm trong Alternate Courses chứ không phải Basic Course. Do đó, guideline #7 nhấn mạnh: một use case chưa hoàn chỉnh nếu chỉ có Basic Course.

Ví dụ:

- “Login” có alternate: forgot password, wrong password, cancel, lock account after 3 fails.
- “Show Book Details” có alternate: book not found.
- “Write Customer Review” có alternate: invalid rating, too short/long review, cancel.

Nếu sơ đồ robustness không thể hiện rõ các alternate này, PDR sẽ phát hiện và đánh dấu đó là lỗi. Đây là một trong những lỗi phổ biến nhất của sinh viên — chỉ tập trung vào luồng chính mà quên xử lý luồng phụ.

## #6. Mỗi use case phải bao phủ cả user + system dialogue

Một use case tốt không chỉ mô tả hành động của người dùng, mà còn phải phản ánh phản hồi của hệ thống. Trong PDR, guideline #6 kiểm tra rằng mọi bước đều có tính đối thoại hai chiều:

- User thực hiện hành động → System phản hồi (thường là controller + boundary hiển thị)

Nếu trong use case hoặc sơ đồ robustness chỉ thấy hành động “người dùng làm X”, mà không có phản ứng hệ thống rõ ràng → đó là một lỗi hổng. Ví dụ: Nếu người dùng nhập sai mật khẩu, hệ thống phải phản hồi bằng ErrorPage hoặc thông báo — nếu không được mô tả, PDR sẽ đánh dấu lỗi.

## #5. Không vi phạm cú pháp robustness

Đây là nguyên tắc kỹ thuật quan trọng. PDR kiểm tra xem sơ đồ robustness có tuân thủ đúng quy tắc Actor–Boundary–Controller–Entity không:

- Actor chỉ nói chuyện với boundary
- Boundary ↔ Controller
- Controller ↔ Entity
- Không có Boundary ↔ Boundary hoặc Entity ↔ Entity

Ví dụ lỗi:

- BookDetailsPage trực tiếp truy cập Book entity mà không qua controller.
- Two entities “Review” và “Book” gửi message cho nhau.

Những lỗi này cho thấy nhóm chưa hiểu đúng bản chất của tương tác, và nếu không sửa sẽ gây rối khi thiết kế sequence diagram sau này

#### #4. Bao gồm cả technical và non-technical trong buổi review

PDR không chỉ là một buổi kỹ thuật. ICONIX khuyến cáo cả hai nhóm — technical và non-technical — phải cùng tham gia. Lý do:

- Khách hàng/marketing giúp xác nhận tính đúng đắn nghiệp vụ.
- Kỹ thuật kiểm tra logic, nhất quán mô hình.

Nếu chỉ có nhóm kỹ thuật, mô hình có thể đúng về mặt logic nhưng lệch về mặt yêu cầu nghiệp vụ. Nếu chỉ có nhóm nghiệp vụ, nhiều lỗi logic sẽ bị bỏ qua.

Ví dụ: Trong “Write Customer Review”, khách hàng có thể nêu ra rằng review phải được kiểm duyệt trước khi đăng — điều này dẫn tới phát hiện entity mới và controller mới. Nếu không có họ trong buổi PDR, nhóm dev có thể bỏ qua chi tiết quan trọng này

#### #3. Đặt use case trong ngữ cảnh object model & GUI

Nhiều lỗi phát sinh do use case viết tách rời khỏi domain model và GUI. PDR buộc nhóm phải xem xét use case trong ngữ cảnh của các màn hình thực tế và các đối tượng miền.

Điều này tránh được tình trạng tên màn hình mơ hồ, hoặc gọi nhầm object.

Ví dụ: Thay vì nói “The system displays a web page”, cần nêu rõ là “The system displays BookDetailsPage, populated with Book entity attributes”.

#### #2. Không trôi sang detailed design

Một lỗi phổ biến là nhóm quá hào hứng, bắt đầu thêm chi tiết như hàm, lớp, API cụ thể vào PDR. ICONIX nhấn mạnh:

“PDR is conceptual. Don’t drift into implementation.”

Nếu đi quá xa vào thiết kế chi tiết, PDR sẽ mất tác dụng kiểm duyệt mô hình ở mức cao.

#### #1. Thực hiện 6 bước kiểm duyệt sơ đồ

Đây là guideline tổng hợp, dẫn trực tiếp đến quy trình PDR 6 bước mà chúng ta sẽ trình bày ở phần kế tiếp.

### 6.6 Quy trình 6 bước kiểm duyệt sơ đồ trong PDR

PDR không phải một buổi thảo luận ngẫu hứng. ICONIX đề xuất một **quy trình kiểm duyệt rất cụ thể**, đảm bảo nhóm đi qua tất cả khía cạnh quan trọng một cách có hệ thống

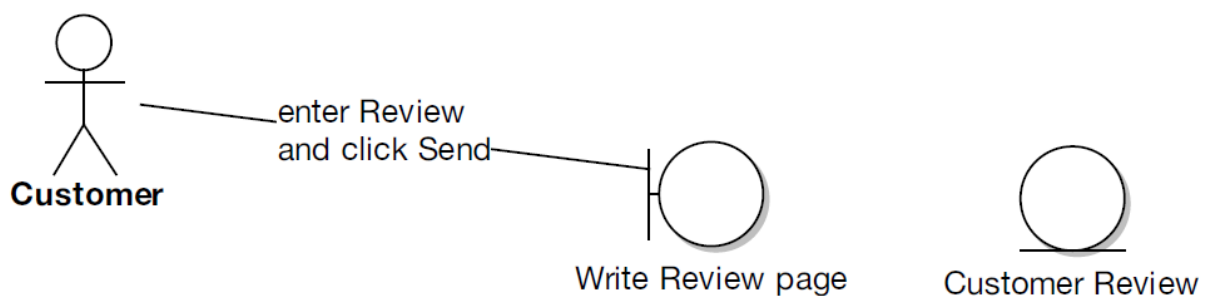
| Bước | Nội dung                                    | Mục tiêu                                          |
|------|---------------------------------------------|---------------------------------------------------|
| 1    | Chạy Highlighter Test                       | Kiểm tra khớp giữa use case và robustness diagram |
| 2    | Kiểm tra alternate flows                    | Đảm bảo alternate được mô tả và xử lý đầy đủ      |
| 3    | So sánh entity giữa diagram và domain model | Phát hiện entity thiếu hoặc trùng                 |
| 4    | Truy vết dữ liệu boundary ↔ entity          | Kiểm tra input/output thuộc tính                  |
| 5    | Kiểm tra quy tắc robustness                 | Phát hiện lỗi cú pháp tương tác                   |
| 6    | Xác nhận stakeholder                        | Đồng thuận từ cả nhóm kỹ thuật và nghiệp vụ       |

Thực hiện quy trình này cho mỗi use case giúp nhóm **loại bỏ mâu thuẫn sớm**, tránh sai sót tốn kém ở giai đoạn thiết kế chi tiết và lập trình.

## 6.7 Ví dụ PDR thực tế: “Write Customer Review”

### 6.7.1. Customer Review object không phải là nơi chứa nội dung đánh giá

Một chủ đề phổ biến trong buổi review là sơ đồ đã không đi vào đủ chi tiết, hoặc bỏ sót các chi tiết quan trọng được mô tả trong use case.



**Figure 6-1.** “enter Review” should be a controller

Reviewer: Khi nhìn vào sơ đồ robustness (xem trích đoạn ở Hình 6-1), tôi mong đợi Customer Review object sẽ là nơi chứa đoạn văn bản “enter review” mà người dùng nhập vào trên màn hình, nhưng hiện tại nó không hề được kết nối.

Analyst: Chúng ta có thể vẽ một đường nối giữa boundary “Write Review page” và entity “Customer Review”...

Reviewer: Nghe thì có vẻ hợp lý, nhưng như vậy sẽ vi phạm quy tắc robustness analysis. Nhớ là, nối hai danh từ (noun–noun) trực tiếp là điều cấm kỵ lớn.

*Ghi chú:* Quy tắc tưởng như khắt khe này có lý do rất rõ ràng: nếu bạn đang nối hai danh từ, thì gần như chắc chắn bạn đã bỏ sót một hành vi (một “động từ”) nào đó cần có controller.

Analyst: Nghĩ lại thì đúng là chúng ta chưa thể hiện rõ hành vi cho hai thao tác của người dùng: nhập nội dung đánh giá và gán điểm rating.

Reviewer: Chúng ta nên thêm các controller để xử lý hai hành vi này, và dùng chúng để nối “Write Review page” với “Customer Review” entity.

Vấn đề được giải quyết! (Phần nhãn “click Send” cũng cần chỉnh lại, nhưng chúng ta sẽ nói ở đoạn sau; Hình 6-2 cho thấy phần sơ đồ đã được cập nhật.)

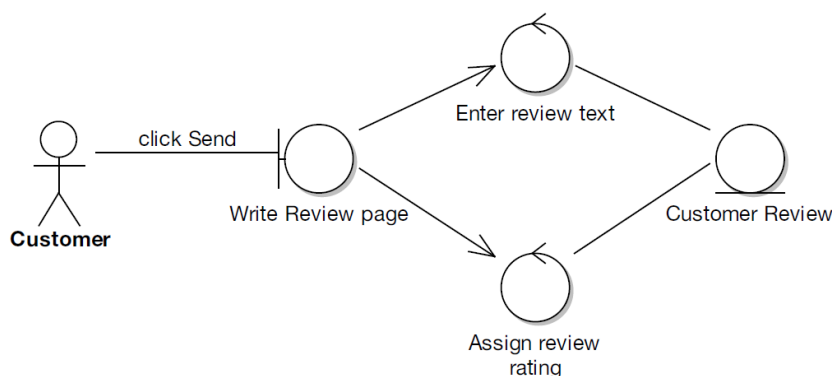


Figure 6-2. Label text is “promoted” to be a controller, and a second controller is identified.

### 6.7.2. Vị trí nhãn mũi tên gây mơ hồ

Khi sửa một lỗi, đôi khi bạn sẽ phát hiện thêm lỗi khác. Sau khi xem xét kỹ hai controller mới được thêm ở Hình 6-2, reviewer nhận ra một rắc rối tiềm ẩn khác. Một phản ứng thường thấy khi được khuyên đưa hành vi nhập dữ liệu vào controller là:

“Nhưng chúng ta đang mô hình hóa web page — những hành vi này được xử lý tự động trong browser, chẳng cần viết code!”

Trả lời: Ở giai đoạn này, bạn không phải đang thiết kế code, mà đang diễn họa toàn bộ văn bản use case thành sơ đồ.

Việc đi vào mức chi tiết như thế có thể hơi tẻ nhạt lúc đầu, nhưng nó thường giúp phát hiện ra rất nhiều chức năng ẩn mà nếu không, sẽ chỉ được phát hiện sau khi thiết kế hoặc tệ hơn — sau khi phần mềm đã chạy và người dùng làm điều “bất ngờ” khiến hệ thống crash.

Bài học: Nếu nó có trong Use Case Text, thì hãy đưa nó lên sơ đồ.

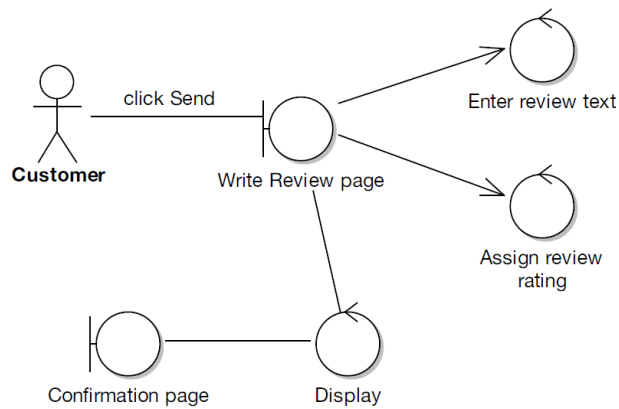


Figure 6-3. The Customer clicked Send... but which way next?

Reviewer: Trong sơ đồ robustness (Hình 6-3), bạn đã dán nhãn “click Send” lên đường từ actor (Customer) đến boundary “Write Review page”.

Analyst: Tôi thấy cũng không có gì sai — ta cần biết người dùng đã làm hành động gì với boundary object.

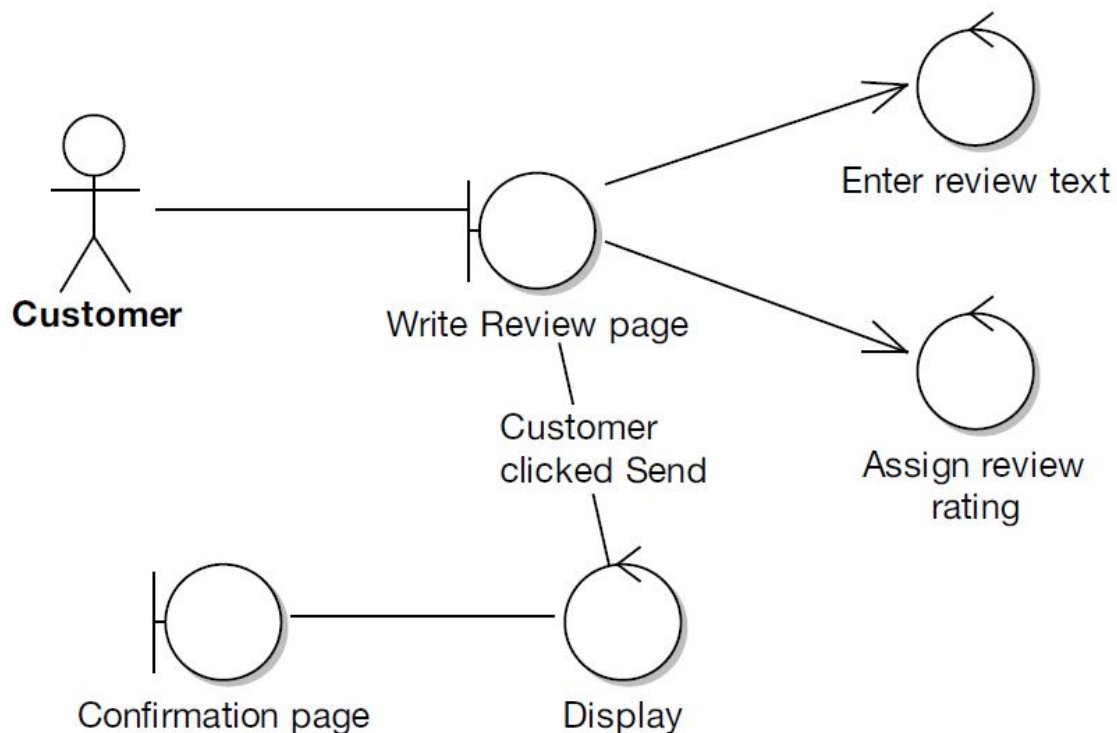
Reviewer: Đúng, nhưng vấn đề nằm ở vị trí nhãn. Khi có nhiều hành động đổ vào cùng một boundary, thì rất nhanh sẽ trở nên mơ hồ xem hành động nào đi tiếp tới đâu.

Analyst: Ý bạn là vì có ba controller xuất phát từ “Write Review page”, nên ta không biết “click Send” sẽ đi đến controller nào?

Reviewer: Chính xác. Sơ đồ sẽ rõ ràng hơn nhiều nếu ta chuyển nhãn hành động sang đoạn giữa boundary và controller.

Nếu bạn gắn “click Send” ở đó, thì sẽ không còn nhầm lẫn controller nào xử lý hành động nào nữa.

(Hình 6-4 minh họa phần sơ đồ sau khi chỉnh sửa.)

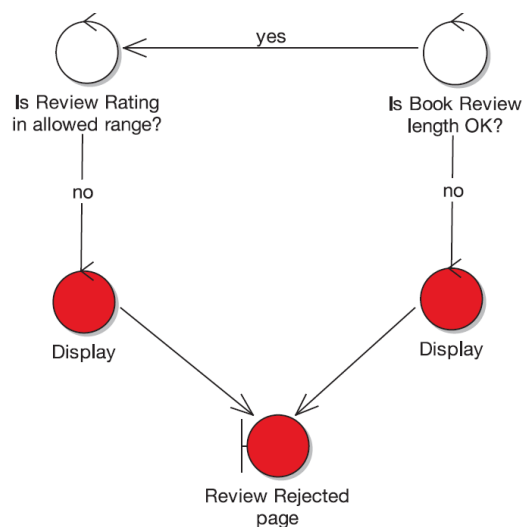


**Figure 6-4.** *Improved label positioning eliminates ambiguity in the diagram.*

### 6.7.3. Controller hiển thị cần tên chi tiết hơn

Khi thêm nhiều controller, đôi khi bạn sẽ lược bỏ bớt chi tiết để đỡ rối. Nhưng điều đó có thể gây ra sự mơ hồ sau này khi vào giai đoạn thiết kế chi tiết.

Reviewer: Bạn đang dùng một boundary “Review Rejected page” chung để thể hiện kết quả của hai controller kiểm tra điều kiện (kiểm tra độ dài review và kiểm tra giá trị rating). Tuy nhiên, mỗi cái có một Display controller riêng (xem Hình 6-5).



**Figure 6-5.** *Insufficient detail on what gets displayed to the user*

Analyst: Tôi tưởng như vậy là đúng rồi?

Reviewer: Đúng, nhưng vấn đề là chúng chưa đủ rõ ràng. Mỗi Display controller sẽ phải tạo một thông báo lỗi khác nhau tùy trường hợp, ví dụ:

“Bài đánh giá của bạn chỉ có 5 ký tự, tối thiểu yêu cầu là 10 ký tự.”

Analyst: Để tôi đoán — ta chưa đặt tên rõ ràng cho các Display controller để thể hiện điều đó.

Reviewer: Chính xác. Hãy đổi tên từng Display controller thành tên mô tả rõ hơn. (Xem Hình 6-6.)

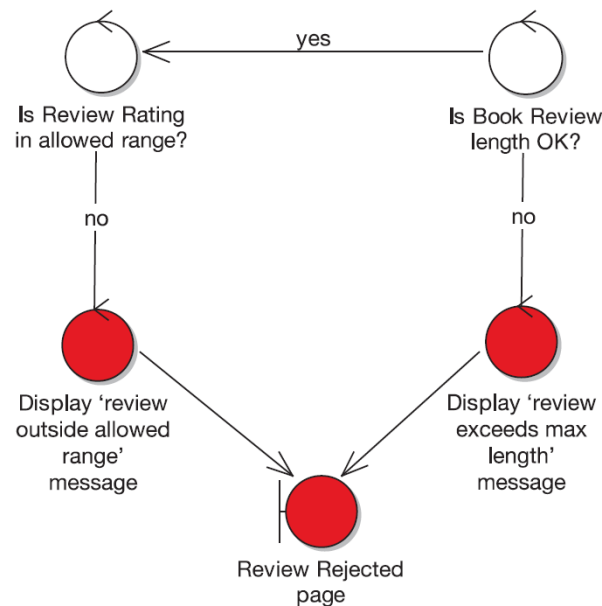


Figure 6-6. Disambiguated Display controllers

#### 6.7.4. Văn bản Use Case cũng đang thiếu chi tiết

Trong PDR, bạn không chỉ kiểm tra sơ đồ mà còn phải kiểm tra chính văn bản use case. Ở ví dụ cuối cùng, reviewer phát hiện một chi tiết bị bỏ sót trong phần mô tả.

Reviewer: Sơ đồ này giờ nhìn rất giống một bản thiết kế có thể chuyển thành chi tiết dễ dàng. Nhưng còn một điểm cuối:

Sơ đồ gọi một use case khác là Moderate Customer Reviews, và nó khớp với đoạn văn bản:

“The system then displays a confirmation screen, and the review is sent to a Moderator, ready to be added.”

Nhưng phần văn bản chưa mô tả chi tiết cách review “đi” tới Moderator như thế nào. Ta đang ngầm giả định rằng review sẽ tự động được chuyển tới Moderator, nhưng điều này không được nêu rõ.

Analyst: Chúng ta có thể sửa văn bản như sau:

“The system then displays a confirmation screen, and the Customer Review is queued for moderation (this will be handled by the Moderate Customer Reviews use case).”

Reviewer: Tốt hơn rồi, nhưng ta cần gắn use case với domain objects. Câu “queued for moderation” chưa nêu rõ đối tượng nào quản lý hàng đợi này.

Analyst: (Suy nghĩ) Có vẻ như ta vừa phát hiện ra một domain object mới: một đối tượng quản lý danh sách Customer Reviews đang chờ.

Reviewer: Hãy gọi nó là Pending Reviews Queue.

Analyst: OK, vậy văn bản sẽ được cập nhật thành:

“The system then displays a confirmation screen and the Customer Review is added to the Pending Reviews Queue for moderation (this will be handled by the Moderate Customer Reviews use case).”

Reviewer: Điều này cũng ngụ ý rằng Book ID nên là một thuộc tính của Customer Review, và điều đó lại ngụ ý rằng Book cũng nên có mặt trên sơ đồ.

Giờ ta có thể thêm Pending Reviews Queue vào sơ đồ.

Analyst: Thật ấn tượng — chỉ cần ràng buộc chặt chẽ giữa văn bản và mô hình mà đã phát hiện ra nhiều chi tiết mới như vậy!

#### 6.7.5. Sơ đồ hoàn chỉnh

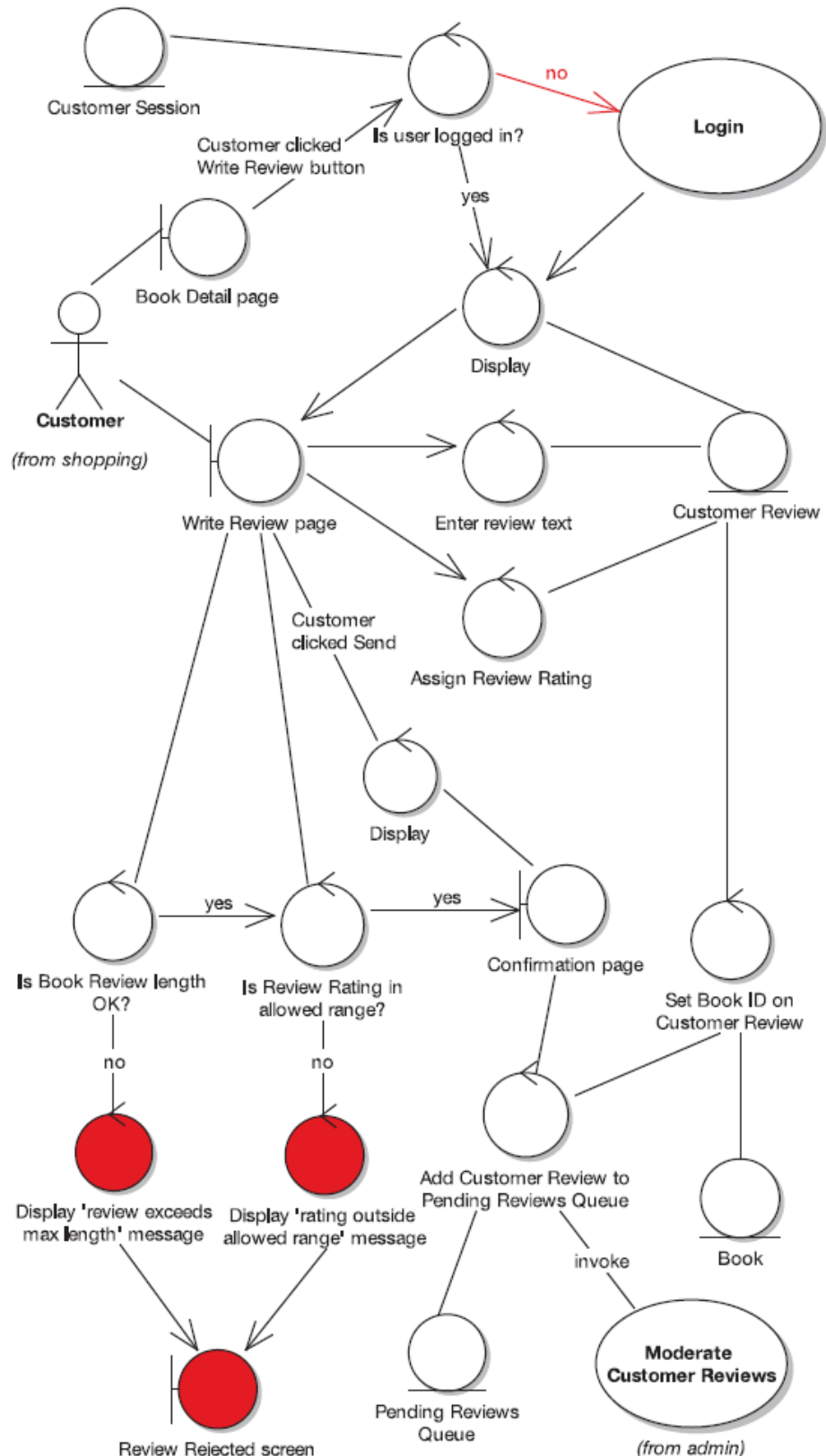
Hình 6-7 cho thấy phiên bản cuối cùng, đầy đủ và rõ ràng của sơ đồ robustness cho use case *Write Customer Review*. Giờ đây, bạn có thể đọc văn bản use case bên trái sơ đồ và lần theo các bước trên sơ đồ từ đầu tới cuối. Không có chi tiết nào bị bỏ sót hay ngầm định, và sơ đồ gắn chặt với domain model. Đây chính là “điểm chốt” mà từ đó ta có thể dễ dàng bước sang giai đoạn thiết kế chi tiết (sequence diagram). Người đọc tinh ý sẽ nhận ra rằng sơ đồ này đã trở nên “rất lớn” — gần đạt giới hạn kích thước use case (theo “quy tắc hai đoạn” ở chương 3). Nếu use case dài hơn nữa, bạn nên tách nó thành nhiều use case nhỏ hơn, mỗi cái có sơ đồ riêng.

**BASIC COURSE:**  
On the Book Detail screen for the book currently being viewed, the Customer clicks the Write Review button. The system checks the Customer Session to make sure the Customer is logged in, and then displays the Write Review screen. The Customer types in a Book Review, gives it a Book Rating out of 5 stars, and clicks the Send button. The system ensures that the Book Review isn't too long or short, and that the Book Rating is within 1-5 stars. The system then displays a confirmation screen, and the review is added to the Pending Reviews Queue for moderation (this will be handled by the Moderate Customer Reviews use case).

**ALTERNATE COURSES:**  
User not logged in: The user is first taken to the Login screen, and then to the Write Review screen once they've logged in.

The user enters a review which is too long (text > 1MB): The system rejects the review, and responds with a message explaining why the review was rejected.

The review is too short (< 10 characters): The system rejects the review.



**Figure 6-7.** The completed, fully disambiguated robustness diagram

## Tổng kết:

Bài giảng 6 đã bao phủ toàn bộ nội dung Chapter 6 — Preliminary Design Review — từ bối cảnh thực tiễn, vị trí trong ICONIX, mục tiêu, Top 10 Guidelines, Highlighter Test, quy trình kiểm duyệt, đến ví dụ thực tế. Đây là bước quan trọng cuối cùng trước khi thiết kế chi tiết, quyết định tính thống nhất và độ “sạch” của mô hình hệ thống.

## Câu hỏi ôn tập

1. Trình bày bối cảnh và vai trò của PDR trong quy trình ICONIX.
2. Nêu 3 mục tiêu chính của PDR.
3. Giải thích Highlighter Test và tại sao nó quan trọng trong PDR.
4. Nêu guideline #9 và lấy ví dụ minh họa thực tế.
5. Vì sao guideline #7 (Alternate Course) đặc biệt quan trọng?
6. Trình bày 5 lỗi phổ biến mà PDR có thể phát hiện.
7. Mô tả quy trình 6 bước kiểm duyệt sơ đồ trong PDR.
8. Trong ví dụ “Write Customer Review”, buổi PDR đã phát hiện và chỉnh sửa những điểm nào?
9. Vì sao PDR cần có cả stakeholder kỹ thuật và phi kỹ thuật?
10. PDR giúp giảm chi phí phát hiện lỗi như thế nào?